

Operating Systems Laboratory - Capstone Projects & Advanced System Simulations

(Faculty Policy Update)

I. Instructor's Guidelines & Evaluation Protocol

Group Policy

- **Group size: 2 students per group.** No exceptions without prior faculty approval.

Mandated Environment (applies to all 18 projects)

To keep grading reproducible and to prevent version-specific dead ends (especially in P5, P17, P18):

Component	Specification
OS / Distro	Ubuntu 22.04 LTS
Kernel	5.15 LTS (as shipped)
Compiler	gcc 11+ / clang 14+
Virtualization	VirtualBox or UTM, provided as a standard lab VM image
Root/privileged work (P2, P5, P17) Must be done inside the VM, never on host or shared lab machines	

Students using a different environment must get it pre-approved; “it worked on my machine” is not an acceptable defense at viva if the grading VM behaves differently.

Safety Mandate (Non-Negotiable)

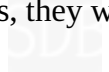
For **Project 2 (Fork Bomb)** and **Project 17 (Kernel Module)**:

- A VM snapshot must be taken immediately before first execution.
- Deliverables must include a screenshot/log of the snapshot being taken, dated before the run.
- Failure to snapshot before a kernel-panic-capable or resource-exhausting run is treated as a lab-safety violation, independent of whether the project itself worked.

The Bonus Mark System

Currently, **there are no mandatory marks assigned for these term projects**; however, it is highly recommended that students complete as many as possible to solidify their practical engineering skills. In the era of AI coding assistants, building the baseline code for these applications is not inherently difficult. The true objective of these projects is to demonstrate deep comprehension of operating system mechanics.

If a student or group demonstrates a profound understanding of the underlying concepts and complexity of a project, and subsequently tackles unsolved challenges or implements novel improvements, they will receive **bonus credit toward their overall term grade**, under the cap below.



Bonus Cap Policy:

Scenario	Cap
Standard bonus performance at viva	Overall term grade may improve by at most one full letter grade (e.g., B → A, C → B, D → C)
Exceptional performance (rare — faculty’s sole discretion, reserved for genuinely novel, near-publication-quality work with flawless defense)	Overall term grade may improve by at most two letter grades (e.g., C → A)
In all cases	The improvement cannot exceed the highest grade on the scale — a two-grade jump from a grade one below the top still only reaches the top grade, not beyond it

This bonus can be used to compensate for shortcomings in other evaluation criteria (e.g., maintaining the lab copy, poor attendance, or low mid-semester performance) **up to the grade-improvement cap above** — it is a cap on outcome, not on effort or viva score, so faculty award only as much of the raw bonus as is needed to reach the applicable capped grade.

Procedure for Claiming Bonus Marks:

1. **The Declaration:** Every submission (baseline or bonus) must include a one-paragraph **Implementation Declaration** stating what was AI-assisted vs. hand-written/hand-debugged, and which parts (if any) the group could not fully explain unassisted. This is not a penalty in itself — it sets viva expectations and creates an audit trail.
2. **The Write-Up:** Students choosing to present a novel problem or improvement must submit a formal write-up detailing their architectural approach and the specific OS concepts utilized.
3. **The Request:** The group must request a discussion time slot to sit for a viva voce (oral defense) with the faculty.
4. **The Viva Voce:** The awarding of marks, and the specific amount given (within the cap above), is at the sole discretion of the teacher based on the student’s performance during the defense.

Difficulty Tiers (for viva time-budgeting and student expectations)

Tier 1 (Intro)	Tier 2 (Core OS)	Tier 3 (Systems-Advanced)
P2 Fork Bomb	P1 Bank Heist	P16 FUSE Filesystem
P3 Mini-Shell	P4 Edge Load Balancer	P17 Kernel Module (LKM)
P10 Defrag Sim	P5 Namespaces	P18 KVM Hypervisor
P15 Signal Handler	P6 Custom Malloc	
	P7 Disk Scheduling	
	P8 IPC Chat (msgqueue)	
	P9 Deadlock Detection Sim	
	P11 Thrashing Sim	
	P12 Threaded Web Server	
	P13 KV Store (R/W problem)	
	P14 Micro-Filesystem	

Tier 3 projects should be allotted longer viva slots and are the only ones eligible for the two-grade “exceptional” bonus tier by default (Tier 1/2 groups can still qualify, but the bar is proportionally higher given the smaller baseline scope).

Examples of Approaches Qualifying for Bonus Marks

- **Project 3 (Mini-Shell):** Supporting complex I/O redirection (`>`, `<`) and multi-stage piping (`cmd1 | cmd2 | cmd3`) by manually managing file descriptors and `dup2 ( )`, **and/or** background process execution via `&`.
- **Project 6 (Custom Malloc):** Implementing a “Buddy System” allocator rather than a simple First-Fit linked list, with a mathematical proof of reduced external fragmentation.
- **Project 12 (Web Server):** Implementing dynamic thread-pool scaling (spawning/killing workers based on active load) instead of a static thread count.
- **Project 16 (FUSE):** Building a transparently encrypted file system (see Project 16 note on cryptographic library requirements below).

The Penalty Trigger (Academic Integrity Warning)

If students attempt to be clever by presenting AI-generated or copied work without understanding the underlying mechanics, thereby wasting the teacher’s time, they will receive a severe grade penalty.

Examples of behaviors that will trigger a penalty during the Viva Voce:

- **Project 1 (Bank Heist):** Submitting a perfectly synchronized POSIX program, but being unable to explain the difference between a mutex and a binary semaphore when asked by the faculty.
 - **Project 5 (Docker’s Roots):** Submitting the working code but failing to explain what the `clone ( )` system call does or what a PID namespace actually isolates.
 - **Project 17 (Kernel Module):** Presenting a working Loadable Kernel Module that hooks a syscall, but being completely unable to trace the pointer logic or explain `copy_to_user`.
-

II. Project Evaluation Rubrics

Evaluation Criteria	Baseline (No Bonus)	Bonus Marks	Penalty Triggers
Conceptual Mastery	Can explain what the code does at a high level.	Can explain exact kernel-level mechanics, memory management, and system calls utilized.	Inability to explain basic OS concepts present in the submitted code.
Implementation	Code compiles and meets the basic expected output.	Code is optimized, handles edge cases, includes robust error handling, and implements a novel feature not explicitly requested.	Code relies entirely on unexplained AI generation or copy-pasting.
Verification Rigor <i>(new)</i>	Output looks correct on a single sample run.	Stress-tested under adversarial thread counts/timing/inputs, with reproducible test logs.	No test methodology — a single “lucky” run presented as proof of correctness.
Novelty & Improvement	N/A (standard project completion).	Identifies a specific bottleneck/limitation in the standard prompt and engineers a custom, measurable solution.	Falsely claiming a standard AI output is a “novel” personal improvement.
Viva Voce Defense	N/A.	Confidently defends architectural decisions, answers deep technical follow-ups, traces execution paths.	Wasting faculty time via lack of preparation or fundamental misunderstanding of the submitted write-up.

III. Capstone Project Directory

(18 projects, deliverables clearly tagged [Baseline] / [Bonus]; per-project faculty notes added where the original spec had a technical or safety gap.)

Project 1: The “Bank Heist” (Defeating Race Conditions)

Description: Students are provided with a multi-threaded banking application that processes thousands of deposits and withdrawals to a shared `bank_balance`. The starter code intentionally lacks synchronization. Students must identify the race condition causing money to “vanish” and implement strict POSIX mutex locks or semaphores to secure the transaction pipeline.

Expected Input: A batch parameter specifying the number of concurrent transaction threads (e.g., 10,000).

Expected Output: A terminal log showing the initial balance, active thread executions, and a mathematically perfectly resolved final balance (zero data corruption).

Deliverables:

- [Baseline] The vulnerable C code demonstrating the corrupted final balance.
 - [Baseline] The secured C code utilizing `<pthread.h>` and `<semaphore.h>`.
 - [Bonus] A 1-page analysis explaining the exact assembly-level instruction interleaving that caused the original race condition.
-

Project 2: The Controlled “Fork Bomb” (Resource Limits & Security)

Description: This project teaches system resource exhaustion and containment. Students will write a program that infinitely spawns child processes using a `while(1) { fork(); }` loop inside an isolated virtual machine. They will then learn to use Linux `ulimit` and `cgroups` to restrict user process limits, effectively neutralizing the bomb.

Expected Input: Execution of the binary before and after applying strict OS-level resource limits.

Expected Output: System crash/freeze (Phase 1). A graceful failure where the OS intercepts and blocks process creation, outputting `Resource temporarily unavailable` (Phase 2).

 **Safety requirement:** VM snapshot mandatory before Phase 1 execution (see Safety Mandate above).

Deliverables:

- [Baseline] The C source code for the fork bomb, plus the snapshot log.
 - [Baseline] A step-by-step terminal script documenting the `ulimit/cgroups` commands used to secure the environment.
-

Project 3: Building a Custom “Mini-Shell”

Description: Students will build their own Command Line Interface (CLI) in C. The shell must parse user input, use `fork()` to spawn child processes, and use the `execvp()` family to execute standard Linux commands (like `ls`, `pwd`, or `cat`). It must also implement the `wait()` system call to prevent the shell from accepting new input until the foreground process finishes.

Expected Input: Standard bash commands typed into the custom `StudentOS>` prompt.

Expected Output: The accurate execution and terminal output of the requested commands, identical to a native Linux bash environment.

Deliverables:

- [Baseline] The C source code implementing the REPL, correctly executing standard commands.
- [Bonus] I/O redirection (`>`, `<`) and multi-stage piping via `dup2()`.
- [Bonus] Background process execution via `&`.

(Moved from a baseline “Advanced implementation” bullet — this is bonus-tier scope per Section I.)

Project 4: Edge Node Load Balancer (CPU Scheduling in Dense Networks)

Description: Applying CPU scheduling to distributed systems. Students simulate an edge computing node receiving thousands of heterogeneous data requests (e.g., lightweight sensor telemetry vs. heavy video processing). They must implement a Priority or Shortest Job First (SJF) algorithm to efficiently clear the local queue and prevent bottlenecking before forwarding tasks to the cloud.

Expected Input: A large CSV file containing 10,000 simulated process requests (Arrival Time, Burst Time, Task Type).

Expected Output: A processed log detailing the execution order, along with aggregate statistics for Average Waiting Time and Turnaround Time.

Deliverables:

- [Baseline] The C source code for the edge load balancer, with a processed execution log and Average Waiting/Turnaround Time statistics.
 - [Bonus] A performance graph comparing SJF/Priority against baseline FCFS.
-

Project 5: Docker’s Roots: Namespaces and Process Isolation

Description: Modern containerization relies heavily on OS primitives. Students will use the Linux `clone()` system call with the `CLONE_NEWPID` flag to create a child process that has its own isolated PID namespace. The child will believe it is PID 1 (the `init` process), demonstrating how tools like Docker isolate applications from the host OS.

Expected Input: Execution of the custom C binary with elevated (root) privileges.

Expected Output: The parent process prints its PID (e.g., 4056). The child process prints its PID, which will output as 1, successfully proving namespace isolation.

Note: Requires root / `CAP_SYS_ADMIN` inside the VM. A permission-denied error here is an environment issue, not a code bug — don't burn debugging time on it; run as root inside the mandated VM.

Deliverables:

- [Baseline] The C source code utilizing `sched.h` and namespace flags, with parent PID and child PID=1 output demonstrated.
 - [Bonus] A brief report connecting this mechanism to container runtimes (Docker/containerd).
-

Project 6: User-Space Memory Allocator (`malloc/free` clone)

Description: Students bypass the standard C library memory allocation and request a large block of raw memory from the OS kernel using the `sbrk()` or `mmap()` system calls. They must then write their own versions of `malloc()` and `free()` to manage this block using a linked list and a First-Fit or Best-Fit contiguous allocation strategy.

Expected Input: Various internal calls to `custom_malloc(size)` and `custom_free(pointer)` within a test program.

Expected Output: Successful allocation and deallocation of memory addresses, with no memory leaks or segmentation faults.

Deliverables:

- [Baseline] `custom_mem.h` + implementation using First-Fit or Best-Fit, with a test suite proving no leaks/segfaults across normal alloc/free patterns.
 - [Bonus] Buddy System allocator implementation with a mathematical proof of reduced external fragmentation (this replaces any baseline expectation of fragmentation-handling proof — that analysis is bonus-tier, not baseline).
-

Project 7: Competitive Algorithm Leaderboard (Disk Scheduling)

Description: A class-wide optimization challenge. Students are provided a massive hidden dataset of randomized magnetic disk track requests. They must implement the most highly optimized C-SCAN (Circular SCAN) or SSTF (Shortest Seek Time First) algorithm possible to minimize total head movement.

Expected Input: A unified dataset of 50,000 disk I/O requests.

Expected Output: The absolute minimum number of total head movements (seek distance) calculated by the algorithm.

Submission contract (new — required for the auto-grading harness):

Input: stdin, one line per request: `<track_number>\n`
first line = initial head position

Output: stdout, single integer = total head movement (seek distance)

Deliverables:



- [Baseline] Optimized C source code for C-SCAN or SSTF conforming to the I/O contract above.
 - [Bonus] Ranking on the class leaderboard against the hidden evaluation dataset.
-

Project 8: Asynchronous Chat Application (Advanced IPC)

Description: Using POSIX Message Queues, students will build a two-way, asynchronous command-line chat application. Two independent terminal windows will run the client binaries. The system must use `msgget()`, `msgsnd()`, and `msgrcv()` to allow real-time communication, utilizing message types to differentiate between standard messages and “typing...” status indicators.

Expected Input: Text strings typed into Terminal A and Terminal B.

Expected Output: Real-time appearance of the text in the opposite terminal, managed entirely through kernel-level IPC queues rather than network sockets.

Deliverables:

- [Baseline] `chat_client.c`, with real-time bidirectional messaging between two terminals.
 - [Baseline] Proper queue destruction (`msgctl + IPC_RMID`) on exit — this is correctness, not bonus (a leaked kernel queue is a bug, not a missing feature).
 - [Bonus] “Typing...” status indicator via a distinct message type.
-

Project 9: Multi-Resource Deadlock Detection Simulation

(Renamed from “Distributed Deadlock Detection Engine” — the described system is a single-process simulation over a shared allocation matrix, not an actual multi-node/networked system. If a group wants the “distributed” version, see Bonus below.)

Description: Extending the Banker’s Algorithm. Students simulate a database cluster where multiple logical nodes hold and request locks on shared tables. The program must periodically scan a central allocation matrix, build a Resource Allocation Graph (RAG), and detect cycle formations (deadlocks) in real-time.

Expected Input: A dynamic stream of lock requests and lock releases from various simulated nodes.

Expected Output: Real-time log granting requests, until a cycle is formed, at which point the system outputs **DEADLOCK DETECTED: Nodes [X, Y, Z] involved in circular wait.**

Deliverables:

- [Baseline] C source implementing Resource Allocation Graph construction and cycle detection over a simulated lock-request/release stream, correctly outputting **DEADLOCK DETECTED: Nodes [X, Y, Z] involved in circular wait.**
 - [Baseline] A test script that intentionally forces a circular wait to prove detection works.
 - [Bonus] Making it *actually* distributed: separate OS processes (or sockets) as the simulated nodes, communicating lock requests over real IPC/network rather than a shared in-memory matrix.
-

Project 10: File System Defragmenter Simulation



Description: Taking contiguous memory allocation further, students simulate a heavily fragmented hard drive. They will be given an array representing disk blocks, with files scattered chaotically due to simulated deletions and creations. They must write a compaction algorithm that shifts all active file blocks to one end of the array, updating the simulated directory pointers, to create a single massive free block.

Expected Input: An array representing a fragmented disk (e.g., [FileA, FileA, FREE, FileB, FREE, FREE, FileC]).

Expected Output: A compacted array (e.g., [FileA, FileA, FileB, FileC, FREE, FREE, FREE]) and an updated directory table showing the new starting block for each file.

Deliverables:

- [Baseline] C source for the compaction algorithm, with a before/after memory map printed to terminal and an updated directory table.
-

Project 11: The Thrashing Simulator (Page Replacement)

Description: A stress-test of virtual memory concepts. Students simulate a system with a very small physical frame count (e.g., 4 frames) but a massively demanding logical reference string representing three concurrent applications. They will run this simulation through both FIFO and LRU algorithms to empirically demonstrate “thrashing” (where page fault rates near 100%).

Expected Input: A highly volatile array of page references and a severely restricted frame constraint.

Expected Output: A terminal graph or tabular output showing the page fault rate skyrocketing as the reference string is processed.

Deliverables:

- [Baseline] C simulation code with a fault-rate table/graph across the reference string.
 - [Bonus] Analytical write-up on how increasing locality of reference would reduce the observed thrashing.
-

Project 12: Multi-Threaded Web Server

Description: Combining OS process concepts with networking. Students will write a basic HTTP web server in C using `<sys/socket.h>`. To handle multiple simultaneous browser requests, the server must implement a thread pool using `<pthread.h>`. When a connection arrives, the main thread dispatches it to a worker thread for processing.

Expected Input: Standard HTTP GET requests sent from a web browser to `localhost:8080`.

Expected Output: The browser successfully rendering an HTML page served by the C program.

Baseline correctness requirement (new): The server must not be trivially exploitable via basic path traversal (e.g., a GET `/../../../../etc/passwd`-style request must not serve files outside the intended web root). This is treated as a baseline correctness bug, not a bonus security feature.

Deliverables:

- [Baseline] C source for the server with a working thread pool and the path-traversal guard above.
 - [Baseline] Proof of concurrency: a stress test with 50 simultaneous connections without dropped packets.
 - [Bonus] Dynamic thread-pool scaling based on active load.
-

Project 13: In-Memory Key-Value Store (Readers-Writers Problem)

Description: Students will build a high-speed, in-memory database (similar to a basic Redis clone) structured around a Hash Table. The challenge is concurrency: multiple threads will attempt to read and write to the database simultaneously. Students must implement the classical Readers-Writers synchronization solution, allowing multiple concurrent readers, but strictly isolated, exclusive access for writers.

Expected Input: Concurrent threads issuing `PUT(key, value)` and `GET(key)` commands.

Expected Output: Accurate data retrieval with zero race conditions, prioritizing read-heavy throughput.

Deliverables:

- [Baseline] C source for the store and threading logic, with correct concurrent `PUT/GET` semantics.
 - [Baseline] Performance metric: time to process 10,000 reads and 1,000 writes.
-

Project 14: Designing a Micro-File System

Description: Students build a functional, miniature file system entirely inside a single 10MB binary file on their hard drive. They must format this 10MB file by defining a Superblock (for volume metadata), an Inode table (for file metadata/pointers), and Data blocks. They will write an interface to `copy_in`, `copy_out`, and `list` files inside their virtual disk.

Expected Input: Commands like `./myfs format`, `./myfs copy_in test.txt`, and `./myfs list`.

Expected Output: The successful storage and retrieval of real text files in and out of their 10MB binary “disk”.

Deliverables:

- [Baseline] C source implementing Superblock/Inode/Data-block logic with working `format`, `copy_in`, `copy_out`, `list`.
 - [Baseline] A hex dump (xxd) of the 10MB file proving byte-level correctness.
-

Project 15: Network Packet Logger & Graceful Shutdown (Signal Handling)

Description: Synthesizing signal trapping and I/O. Students write a program that opens a raw socket to continuously sniff and log incoming network packets to a text file. The primary challenge is data integrity: if the user presses `Ctrl-C`, the program must trap the `SIGINT` signal, stop accepting new packets, explicitly flush the I/O buffers, safely close the file pointers, and then exit gracefully without corrupting the log.

Expected Input: Continuous network traffic, interrupted suddenly by a keyboard signal (`SIGINT`).

Expected Output: A perfectly formatted packet log file that cuts off cleanly exactly when the signal was received, with no corrupted or half-written binary data at the end of the file.

Deliverables:

- [Baseline] C source with the custom signal handler and I/O flushing logic, producing a clean log file with no corrupted trailing data on `Ctrl-C`.
-

Project 16: User-Space File System (FUSE) Implementation

Description: Students will utilize the FUSE (Filesystem in Userspace) library to create a custom, mountable file system where file operations are intercepted and handled by their own C code. As a novel challenge, they can build a “RAM-only” file system or an “Encrypted” file system that transparently encrypts data upon write and decrypts upon read.

Expected Input: Standard bash commands (`touch`, `mkdir`, `echo`, `cat`) executed inside the mounted FUSE directory.

Expected Output: The virtual file system behaves exactly like a native directory, but the underlying data storage is managed dynamically by the student’s background process.

Cryptography requirement (new): If attempting the encrypted-FS bonus variant, students **must use a vetted library primitive** (OpenSSL EVP API or libsodium) — hand-rolled AES implementations are explicitly disallowed. The learning objective is I/O interception and transparent encrypt/decrypt plumbing, not cipher design; home-rolled crypto is bad practice even pedagogically.

Deliverables:

- [Baseline] FUSE source with working `getattr/readdir/read/write`, demonstrated via `mount` → file ops → `unmount`.
 - [Bonus] RAM-only or transparently encrypted variant (per the crypto requirement above).
-

Project 17: Custom Kernel Module for Syscall Interception

Description: Moving beyond user-space, students will write a Loadable Kernel Module (LKM) for Linux. The objective is to hook into the system call table and intercept a basic syscall (such as `sys_open` or `sys_mkdir`), replacing it with a custom wrapper that logs the PID of the calling process before executing the original kernel function.

Expected Input: Loading the compiled `.ko` file using `insmod`, followed by any standard system action that triggers the hooked syscall.

Expected Output: Real-time kernel logs visible via `dmesg` proving that the student's module successfully intercepted the OS-level instruction.

⚠ Technical note (critical): `sys_call_table` has **not been exported** since Linux kernel 4.17+ due to kernel hardening. The original spec's literal approach ("hook into the system call table") will not compile/link out-of-the-box on the mandated 5.15 LTS kernel. Two supported paths:

1. **Recommended baseline approach:** Use **ftrace-based hooking** (via `register_ftrace_function / ftrace_ops`), which works on modern kernels without needing the exported symbol table.
2. **Alternative (only with prior faculty approval):** Build and test against a provided older kernel VM image (e.g., 4.4 LTS) where the classic `sys_call_table` approach still applies directly.

⚠ Safety requirement: VM snapshot mandatory before `insmod` (see Safety Mandate above) — a bad hook panics the kernel.

Deliverables:

- [Baseline] LKM source + Makefile using the ftrace approach (or approved legacy-kernel approach), with `dmesg` logs proving PID interception.
 - [Bonus] A security analysis discussing syscall-hooking implications (e.g., how rootkits operate).
-

Project 18: Virtual Machine Monitor (Hypervisor) Basics

Description: Students will utilize the Linux KVM (Kernel-based Virtual Machine) API `/dev/kvm` to build a rudimentary Type-2 Hypervisor. They must initialize a virtual CPU (vCPU), allocate virtual memory, and load a tiny, custom payload of 16-bit x86 assembly code into the VM to execute.

Expected Input: Execution of the C hypervisor binary, passing in the compiled 16-bit assembly payload.

Expected Output: The hypervisor successfully handles `VMEXIT` conditions (such as the guest OS attempting to output to a serial port) and prints the guest's output to the host terminal.

Prerequisite note (new): Requires reading/writing 16-bit real-mode x86 assembly. This is **not** assumed prior knowledge from the standard CSE curriculum unless a systems/assembly course precedes this lab. To keep the focus on hypervisor mechanics (vCPU setup, memory mapping, `VMEXIT` handling) rather than assembly authorship:

- A **reference 16-bit assembly payload will be provided** by the instructor.
- Students may substitute their own payload only as a bonus variation, not as a baseline requirement.

Deliverables:

- [Baseline] C source using the KVM ioctls to run the provided payload, correctly handling `VMEXIT` (e.g., guest serial-port output) and printing it to the host terminal.
 - [Bonus] A student-authored assembly payload beyond the provided reference one.
-

IV. Implementation Declaration (Template — attach to every submission)

Group Members: _____ Project #: _____

1. Which parts of this submission were AI-assisted (tool + what it generated)?

2. Which parts were entirely hand-written/hand-debugged?

3. Is there any part of the submitted code the group cannot fully explain unassisted? (Y/N – if Y, specify which part)

Signatures: _____